

Recap

Q Learning

Beh Policy/ Tgt Policy

Deep Q Network

Replay Buffer

Stable Baseline (SB3)

On Policy/ Off Policy

SARSA

Actor - Critic

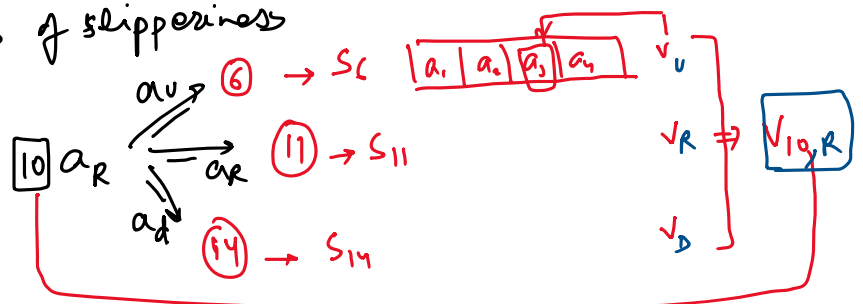
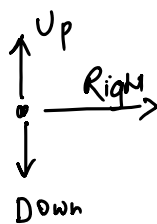
TRPO, PPO, GRPO

Model based

Model Free

Model  $\rightarrow$  Environment  
(Knowledge of environment)

Probabilities of slipperiness



$$\max (V_{10,L} | V_{10,D} | V_{10,R} | V_{10,U}) \rightarrow V_{\max}$$

No knowledge (No idea about Probabilities)

|          | $a_0$ | $a_1$ | $a_2^R$ | $a_3$ |
|----------|-------|-------|---------|-------|
| $s_0$    |       |       |         |       |
| $s_1$    |       |       |         |       |
| $\vdots$ |       |       |         |       |
| $s_{10}$ |       |       |         |       |
| $s_{14}$ |       |       |         |       |

| episode     | Steps                |
|-------------|----------------------|
| past values | Next_Step            |
| $s_0$       | <input type="text"/> |
| $s_{10}$    | <input type="text"/> |
| $s_{14}$    | <input type="text"/> |

$S_{15}$  |  $S_{14}$  |  $S_{13}$  |  $S_{12}$  |  $S_{11}$  |  $S_{10}$  |  $S_9$  |  $S_8$  |  $S_7$  |  $S_6$  |  $S_5$  |  $S_4$  |  $S_3$  |  $S_2$  |  $S_1$  |  $S_0$

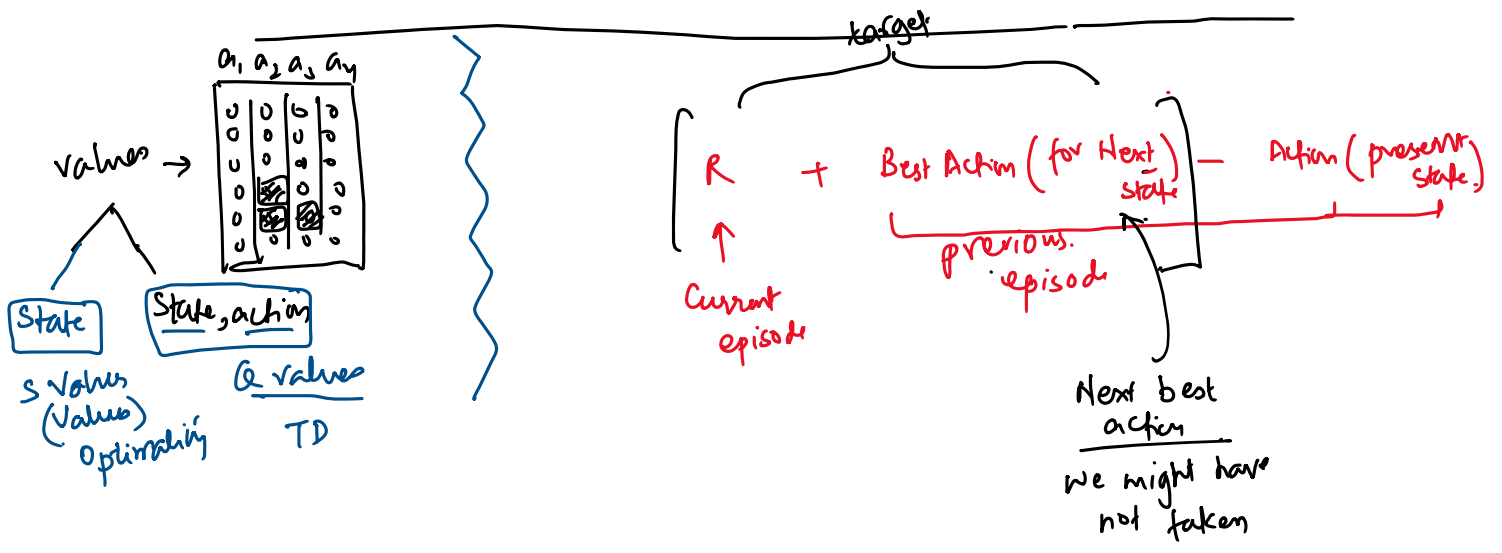
# Bellman Eq of Temporal Difference.

$$Q_{10,R}^{New} = Q_{10,R}^{prev} + \alpha [TD]$$

$$Q_{10,R}^{New} = Q_{10,R}^{Current} + \alpha \left[ \underset{\substack{\uparrow \\ \text{Aim Best Action.}}}{\text{target}} - \underset{\substack{\uparrow \\ \text{Exploration}}}{Q_{10,R}^{Current}} \right]$$

$$Q_{10,R} = Q_{10,R} + \alpha \left[ R + S_{11} \overset{\text{max}}{\boxed{2}} - Q_{10,R} \right]$$

$$Q_{10,R}^{Current\ Episode} = Q_{10,R}^{past\ Ep} + \alpha \left[ R + \underset{\substack{\uparrow \\ \text{Current}}}{\gamma Q_{11,D}^{Next\ Step\ from\ prev\ episode\ past\ Ep}} - Q_{10,R}^{past\ Episode} \right]$$



policy  $\leftrightarrow$  Behaviour (Exploration - Exploitation)   
 Target (Dream) (  $\boxed{\text{max}}$  )

ex. Cart-Pole

## Big Environment

ob's space  $[-1 \dots \dots \dots +1]$   
 $\uparrow$   
 continuous

Cart Pole observation space = 4 ~~Discrete~~ Range  $[-1, +1]$

~~Q learning~~  $\Rightarrow$  Deep Q learning

~~Q learning~~  $\Rightarrow$  Deep Q learning

## Model Based

Model free.

Small Env.      Big Environment

Q Learning      Deep Q Learning

# Deep Learning (Neural Networks)

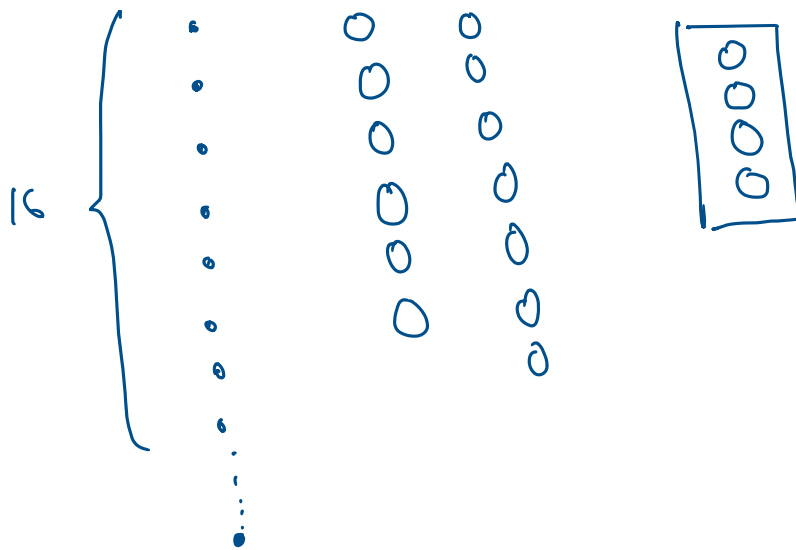
Inputs || || || Output  
(Observation Space) (Action Space)

frozen lake

## Cart - Pole

1

0



$$Q(S, \text{Left}) = 5$$

$$Q(S, \text{Right}) = 10$$

Behavior policy:

$S \rightarrow \text{Left}$  ← actually happened

Target policy:

$S \rightarrow \text{Right}$  ← policy being learned

Q-learning → "What if I take the best action next?"

SARSA → "What action did I actually take next?"

Off-policy RL becomes much clearer if you remember these two terms:

### 1. Behavior policy

The policy that **generates experience**.

$$\pi_b$$

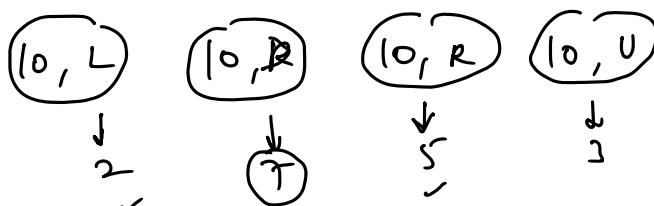
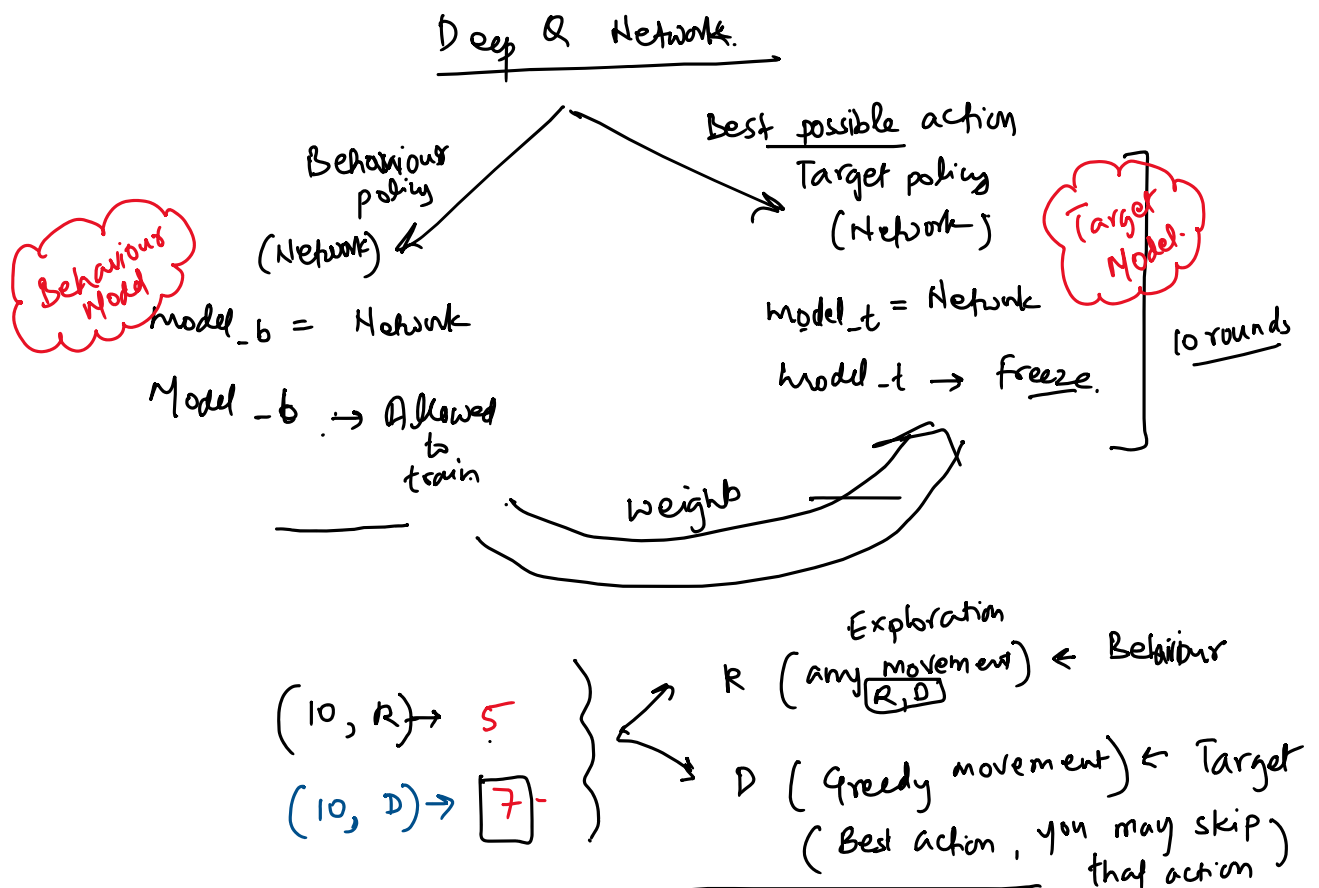
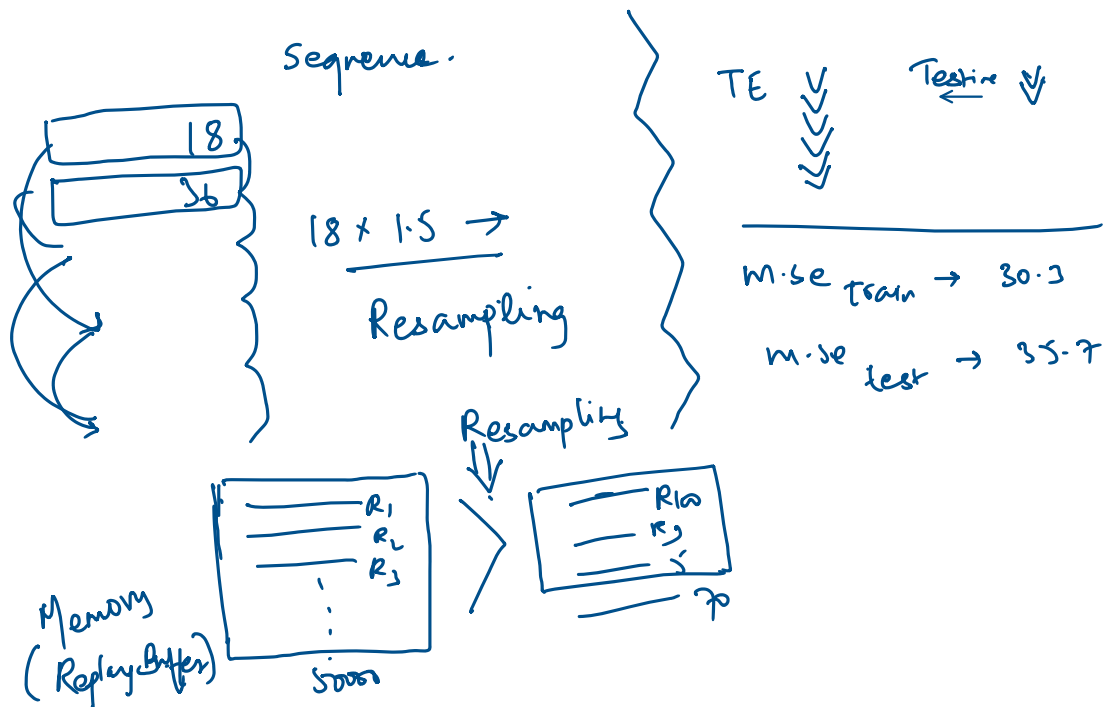
### 2. Target policy

The policy that we are **trying to learn**.

$$\pi_t$$

For off-policy learning:

$$\pi_b \neq \pi_t$$



Q values are going to be shaped by this

$$Q = Q + \alpha \left[ \underbrace{R + \gamma Q_{n_x}}_{\text{target}} - \underbrace{Q}_{\text{Behaviour}} \right]$$

$$Q = Q + \alpha [TD_{err}]$$

12  
10  
...  
0

```
# Choose action (epsilon-greedy)
if np.random.rand() < EPSILON:
    action = env.action_space.sample() # Random action
else:
    with torch.no_grad():
        action = torch.argmax(policy_net(torch.tensor(state)))

# Take action
next_state, reward, done, _, _ = env.step(action)
```

```
# Compute Q-values
q_values = policy_net(states).gather(1, actions)

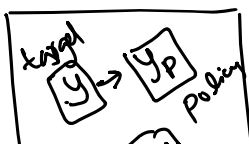
# Q(s,a) = Q(s,a) + alpha * [Reward + gamma * max(Q(s',a')) - Q(s,a)]
# Compute target Q-values using the target network
next_q_values = target_net(next_states).max(1, keepdim=True)[0] # values, indexes
target_q_values = rewards + (GAMMA * next_q_values * (1 - dones))

# Compute loss
loss = loss_fn(q_values, target_q_values.detach())
```

y - y<sub>p</sub>  
y<sub>p</sub> - y

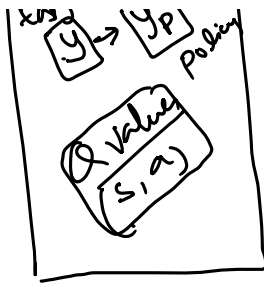
model(x) → y<sub>p</sub>      model.predict(x) → y<sub>p</sub>

$$TD = (\text{target} - \text{q values})$$



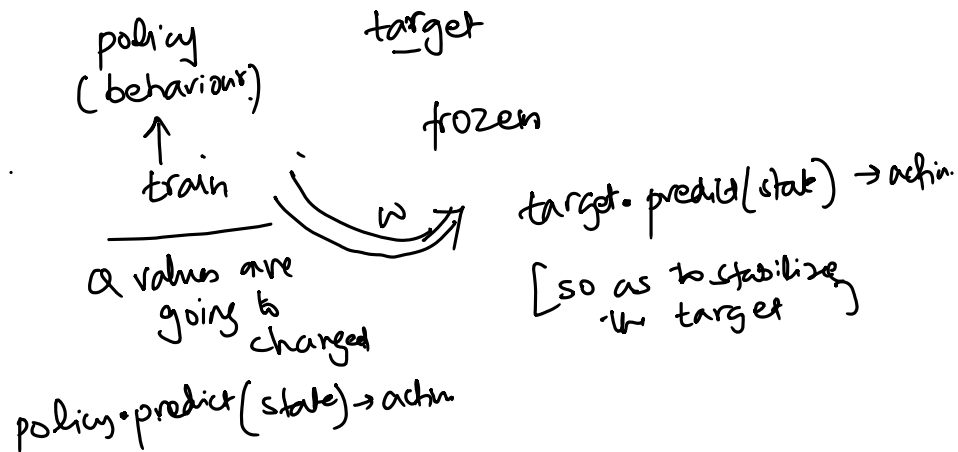
model(state) → action

model.predict(state) → action



model (state) → action

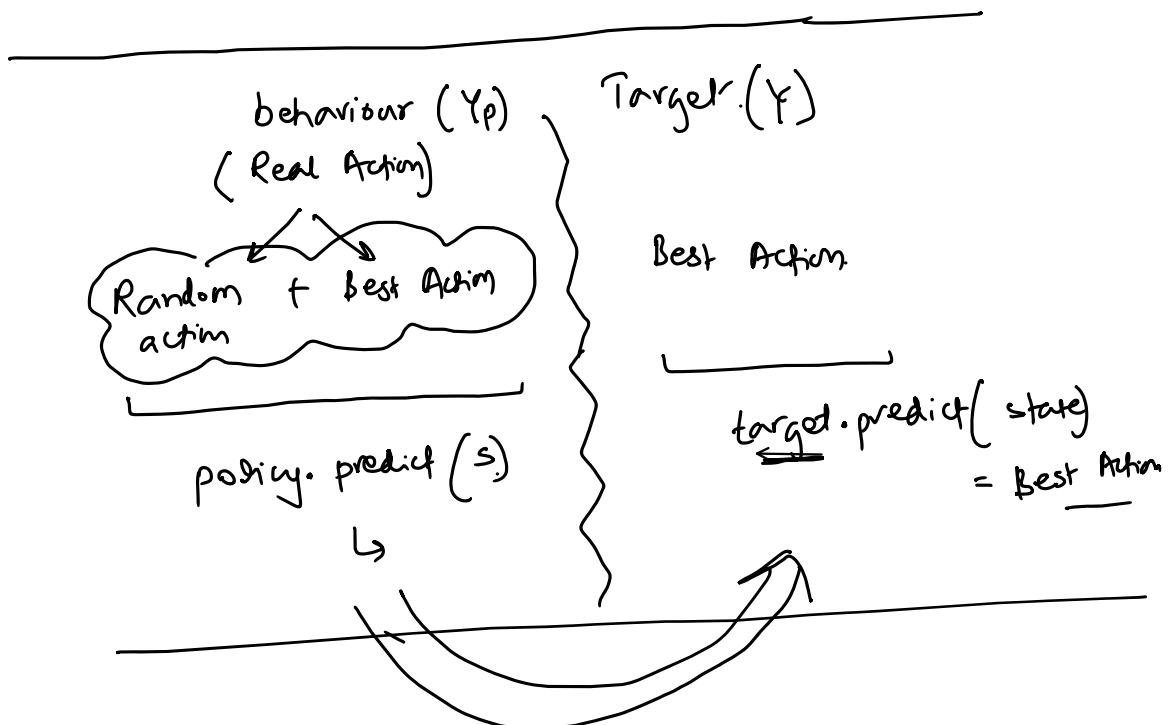
model.predict(state) → action

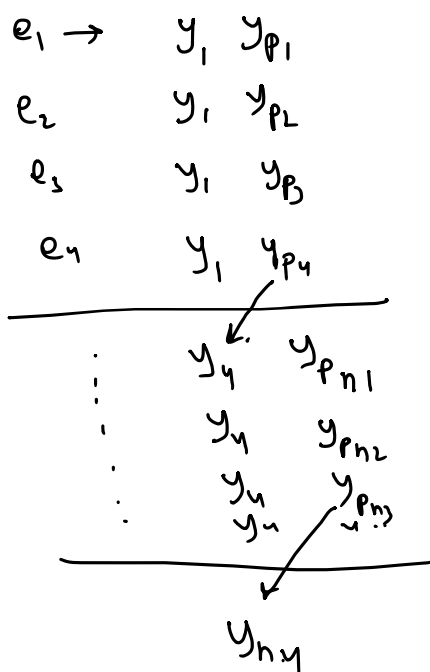
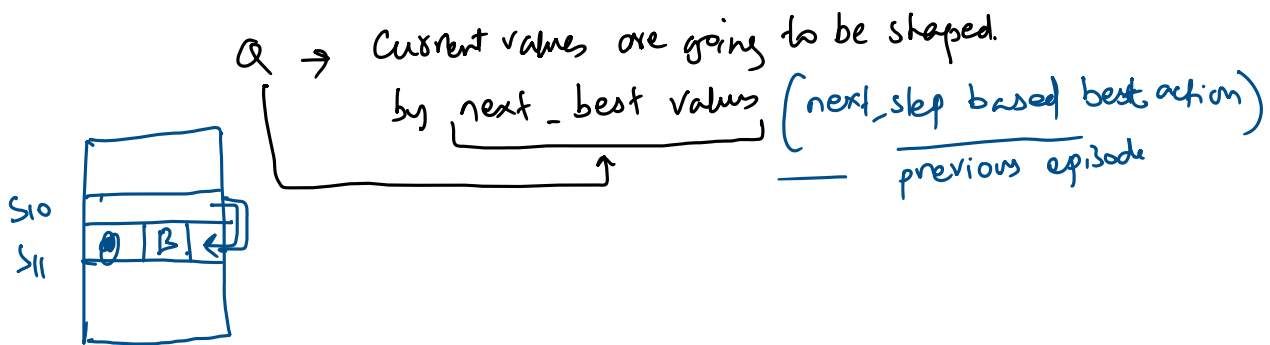


policy.predict(s) = A → y\_p → 'expected output'  
 target.predict(s) = A → y → target

$$TD_e = y - y_p$$

↓                      ↓  
 target - Real Result  
 target.predict(s) - policy.predict(s)





SB-1 → TensorFlow  
SB-3 Library

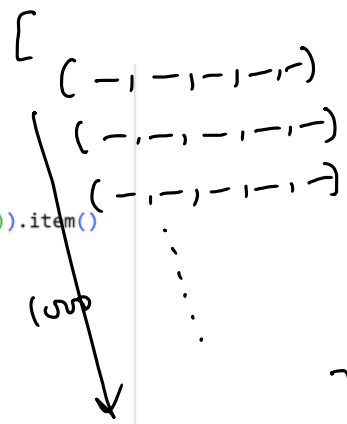
```
memory = []

# Choose action (epsilon-greedy)
if np.random.rand() < EPSILON:
    action = env.action_space.sample() # Random action
else:
    with torch.no_grad():
        action = torch.argmax(policy_net(torch.tensor(state))).item()

# Take action
next_state, reward, done, _, _ = env.step(action)
next_state = np.array(next_state, dtype=np.float32)

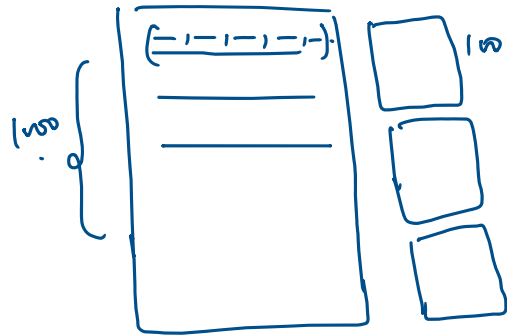
# Store experience
memory.append((state, action, reward, next_state, done))
state = next_state

# Train only if enough samples are in memory
if len(memory) >= BATCH_SIZE:
    batch = random.sample(memory, BATCH_SIZE) # Sample batch
```



Sequence  
Correlation

→ Overfitting



[ state, next\_state, action, done, reward ]

$$Q = Q + \alpha [ R + \gamma Q_{\text{next}} - Q ]$$

new value

target • predict ( )

policy • predict ( )

Policy Based (probabilistic)

Not Week

REINFORCE

Actor-

Actor-Critic

PPO, DPO

TRPO

GRPO

Chat GPT

Deep Seek

Drones

Recap

- Behaviour and Target
- On Policy vs Off Policy
- SARSA [on Policy]

~~Advantage Function~~

Policy Gradient

Reinforce

Advantage Function ✓

Actor-Critic

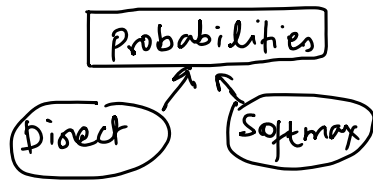
[10, 9, 8]

Without Softmax

[0.37, 0.33, 0.30]

With Softmax

[0.67, 0.24, 0.09]

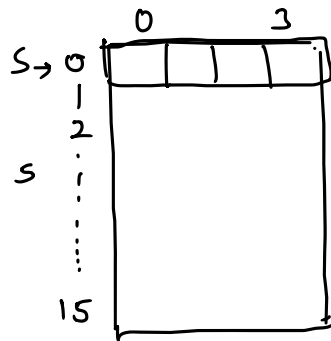
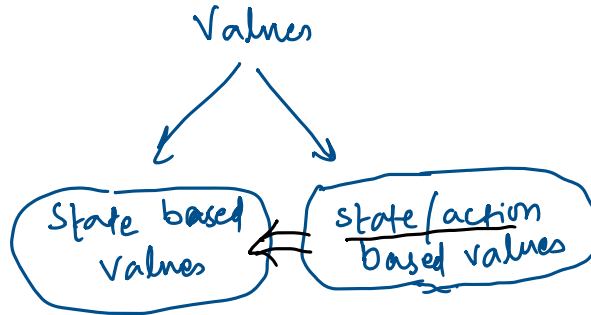


$$[10, 20, 30]$$

$$\downarrow$$

$$\frac{e^{10}}{e^{10} + e^{20} + e^{30}} =$$

Q values ?



$\rightarrow \max(\text{values}) \rightarrow$  value based thing.

State values  $\rightarrow$  BE optimality

Q values  $\rightarrow$  BE Temporal Difference

$$\text{Old values} + \alpha (\text{Target} - \text{Old values})$$

Reward + Old value next

$c \leftarrow$  Old new

Q-learning

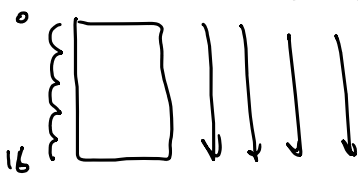
$\rightarrow$  New Value = Old Value +  $\alpha \times (\text{What just happened} - \text{Old Value})$

Every update tweaks the numbers slightly to make the Q-table a more accurate reflection of the true long-term average reward of the environment.

$s \leftarrow$  Curr Next (old)

Target

Behaviour



$$Q(s, A) = Q(s, A) + \alpha [\text{Target} - Q(s, A)]$$

Prev. Episode

$$R + \gamma \max_a Q(s_{\text{next}}, a)$$

$$R + \gamma \max_a Q(s_{next}, a)$$

Behaviour  $80\% + 20\% \rightarrow \epsilon$ -greedy

Target  $\rightarrow 100\% \rightarrow$  Greedy

Policy = how actions are selected

Behavior = actions actually selected

State  $s$   
 $\downarrow$   
 Q-values:  
 $A1 = 10$   
 $A2 = 7$   
 $A3 = 4$   
 $\downarrow$   
 $\epsilon$ -greedy policy  
 $\downarrow$   
 $90\% \rightarrow$  choose  $A1$   
 $10\% \rightarrow$  explore  $A2/A3$

$\epsilon$ -greedy = 10%

greedy

Step 1:  $S1 \rightarrow A1$   
 Step 2:  $S2 \rightarrow A1$   
 Step 3:  $S3 \rightarrow A3 \leftarrow$  exploration  
 Step 4:  $S4 \rightarrow A1$   
 Step 5:  $S5 \rightarrow A2 \leftarrow$  exploration

$S1 \rightarrow A1$   
 $S2 \rightarrow A1$   
 $S3 \rightarrow A1$

1.5 | 2.8 | 1.4 | 1.7

Target (Highest Action)

Off-policy  
(Q-learning)

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_a Q(s', a) - Q(s, a)]$$

On Policy  
(SARSA)

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

behaviours

(Real Action exploration / greedy)

Q-learning: behavior action, target-policy future

next state

SARSA: behavior action, behavior-policy future

next state  
next action

```

# E-greedy policy
def choose_action(state):
    if np.random.uniform(0, 1) < epsilon:
        return env.action_space.sample() # getting a random action
    return np.argmax(Q[state, :])

# Train with SARSA
for ep in range(episodes):
    state = env.reset()
    action = choose_action(state)

    for step in range(max_steps):
        next_state, reward, done, _ = env.step(action)
        next_action = choose_action(next_state)

        # SARSA update
        Q[state, action] += alpha * (reward + gamma * Q[next_state, next_action] - Q[state, action])
        state, action = next_state, next_action

    if done:
        break

# Update Q-value
Q[state, action] = Q[state, action] + alpha * (reward + gamma * np.max(Q[next_state]) - Q[state, action])

```

Q | sarsa

all of action

action in real

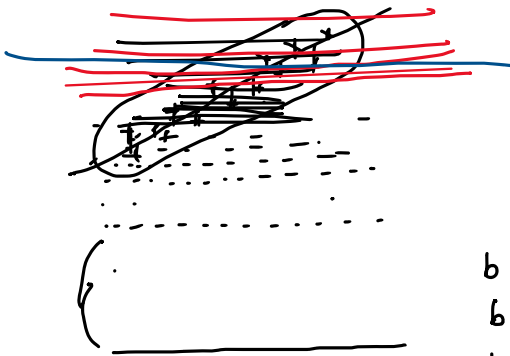
Best Action

Q(14) Q(15) [ 1 1 ]

Off-Policy [target ≈ Dream]  $\xrightarrow{\text{applied}}$  easy terrains (foot-ball Grounds)

On-Policy [target ≈ Reality]  $\xrightarrow{\text{applied}}$  difficult terrains (Walking on hilly areas)

| Value                                                                                                                                                         | Policy                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>State only values</p> <p>State + action values</p> <p>action = <math>\text{np.argmax}(\text{values})</math></p> <p>policy = Collection of all actions.</p> | <p>policy() <math>\rightarrow</math> A<sub>random</sub>.</p> <p>↓</p> <p>Improve after improve in loops <math>\rightarrow</math> A<sub>best</sub></p> <p>gradient of policy</p> |



$w$    
 $b$  



$b = b + \boxed{\text{Error}}$   
 $b = b + \boxed{\phantom{0}}$   
 $b = b + 0$   
 $b = b + \cdot$  ← never be equal to zero  
 $b = b + 0$  ← should be equal to zero  
 $b = b + \boxed{\phantom{0}}$   
 $b = b + \boxed{\phantom{0}}$

$$b = b + \nabla e$$

$$\left. \begin{array}{l} b = b + 0.2 \\ b = b + 0.2 \end{array} \right\} \Delta g \Rightarrow$$

$$\begin{array}{l} b = b + \nabla e \\ b = b + 0 \end{array}$$

Action → Discrete → Value based.  
[L, D, R, L]

Action\_Space → Continuous → Policy based

Policy   
 → [R, R, L, R, R, L, R, L, L, R] ← Deterministic Policy  
 → [0.6, 0.1, 0.7, 0.1, ...] ← Stochastic Policy.  
 Right

If traffic light is red → Stop  
 If traffic light is green → Go

← Deterministic

If traffic light is yellow:  
 Accelerate with 70% chance  
 Brake with 30% chance

← Stochastic Nature.  
(Probability Distributions)

Discrete Continuous

0.7, 0.3

Q-Learning usually ends up with a deterministic policy:

$$\pi(s) = \arg \max_a Q(s, a)$$

$$\text{Prob}(a|s_t)$$

Q-Learning usually ends up with a deterministic policy:

$$\pi(s) = \arg \max_a Q(s, a)$$

$$\text{Prob}(a|s)$$

Policy Gradient, PPO, and many Actor-Critic methods often learn stochastic policies:

$$\pi(a|s)$$

which output action probabilities via softmax.

$$\begin{matrix} s_0, s_1, s_2, s_3, s_4 \\ \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ P \times P \times P \times P \times P \end{matrix}$$

$T_D(0)$   $\xrightarrow{T_D(\lambda)}$   $T_D(1)$  [Monte Carlo]  
every single step      every complete episode.

Monte-Carlo

$\gamma \rightarrow$  Instead of (100%) full striking  
(90%) striking

|          |    |    |    |    |
|----------|----|----|----|----|
| States:  | S0 | S1 | S2 | S3 |
| Actions: | A0 | A1 | A2 | A3 |
| Rewards: | 1  | 1  | 1  | 10 |

$$G_n = 1 + \gamma G_{n+1}$$

$$\begin{aligned} - G_3 &= 10 \\ - G_2 &= 1 + \gamma(10) \\ - G_1 &= 1 + \gamma(1 + \gamma(10)) \\ - G_0 &= 1 + \gamma(1 + \gamma(1 + \gamma(10))) \end{aligned}$$

$$\begin{aligned} G_1 &= 1 + \gamma + \gamma^2 10 \\ G_1 &= 1 + \gamma(1 + \gamma 10) \\ G_1 &= 1 + \gamma G_2 \end{aligned}$$

policy supposed to be improved.

$$\nabla \text{policy. (probabilistic nature)} \rightarrow \nabla \frac{\text{prob}(a|s)}{\pi(a|s)}$$

$$\begin{aligned} w &= w + d\epsilon / dw \\ w &= w + \nabla w \end{aligned} \quad \left| \quad \begin{aligned} \pi &= \pi + d\epsilon / d\pi \\ \pi &= \pi + \nabla \pi \end{aligned}$$

$$\pi_t = \pi_1 \times \pi_2 \times \pi_3 \times \pi_4$$

(track the total policy)  $\pi P = p_1 \times p_2 \times p_3 \times p_4$   
 $\nabla \times \nabla \times \nabla \times \nabla \rightarrow$  Diminish Very high

$$P \rightarrow \log(P)$$

$$\boxed{P \times P \times P} \rightarrow \log(P) + \log(P) + \log(P)$$

$$\frac{d(\log x)}{dx} \rightarrow \frac{1}{x}$$

$$\nabla \log \pi = \frac{1}{\pi}$$

$$\nabla \log K = \frac{1}{K} \nabla K$$

$$\nabla_{\theta} \pi_{\theta}(a|s) = \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s)$$

$$\nabla \log \pi = \frac{1}{\pi} \nabla \pi$$

$$\nabla \pi = \pi \nabla \log \pi$$

$$\pi \rightarrow \pi(a|s) = \text{Prob}(a | s_0)$$

$$\nabla \pi = \pi \nabla \log \pi$$

$$\nabla \pi(a|s) = \pi(a|s) \nabla \log \pi(a|s)$$

$$W = W + \nabla W$$

$$P = P + \nabla P$$

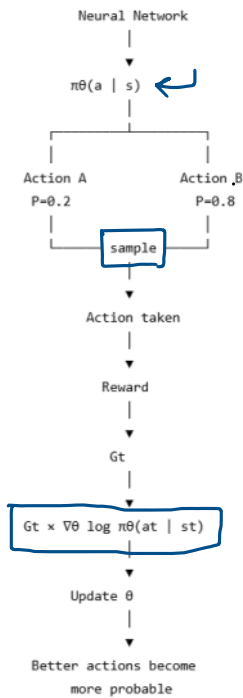
$$G_5 = \sum R$$

$$R_5 + R_6 + \dots + R_{\text{last}}$$

$$R_5 + \gamma G_5$$

$$G_t \times \nabla \rightarrow \text{Big value} \rightarrow \text{Likely to take same route.}$$

$$G_t \times \nabla \downarrow \rightarrow \text{Small value} \rightarrow \text{Less likely to take the same route.}$$



$$ls = [0.2, 0.8]$$

$$ls = \text{sample}() \rightarrow \begin{matrix} 0.2 \\ 0.8 \\ 0.2 \\ 0.8 \\ 0.8 \\ 0.8 \\ 0.2 \\ 0.8 \\ 0.8 \\ 0.8 \end{matrix}$$

$$ls = \begin{bmatrix} 2 \\ 4 \end{bmatrix} \rightarrow \begin{matrix} \downarrow & \downarrow \\ v_3 & z_3 \end{matrix}$$

$$\text{sample}([0.25, 0.75])$$

$$\rightarrow \begin{matrix} 4 \\ 2 \\ 2 \\ 2 \\ 4 \\ 4 \\ 5 \\ 5 \\ 5 \\ 2 \end{matrix}$$

"If I took this action and it turned out well, make this action more likely next time. If it turned out badly, make it less likely."

```

def act(self, state):
    probs = self.forward(state)
    print(probs)
    dist = torch.distributions.Categorical(probs)
    print(dist)
    action = dist.sample()
    return action.item(), dist.log_prob(action)
  
```

Softmax [10, 40]

$$[0.2, 0.8]$$

$$\log[p_1 \times p_2 \times p_3]$$

$$\log p_1 + \log p_2 + \log p_3$$

```

state, info = env.reset()
log_probs, rewards = [], []
done = False
while not done:
    state_t = torch.tensor(state, dtype=torch.float32)
    action, log_prob = policy.act(state_t)
    next_state, reward, terminated, truncated, info = env.step(action)
    done = terminated or truncated

    log_probs.append(log_prob)
    rewards.append(reward)
    state = next_state
  
```

```

# Compute discounted returns
returns = []
G = 0
for r in reversed(rewards):
    G = r + gamma * G
    returns.insert(0, G)
returns = torch.tensor(returns, dtype=torch.float32)
returns = (returns - returns.mean()) / (returns.std() + 1e-8) # normalize
  
```

# Policy gradient update

```

loss = []
for log_prob, Gt in zip(log_probs, returns):
    loss.append(-log_prob * Gt)
loss = torch.stack(loss).sum()
  
```

-ve and converting this as loss

$$a = [2]$$

$$a.item() = 2$$



$$\gamma = 0.9$$

G [10] → total rewards till the episode is finished

$[G_{10} \times \log\_Prob] \rightarrow \underline{\text{Optimization}}$  [to find the best route]

Loss = - Optimization

$$\log(\text{Prob}) \rightarrow -ve.$$
$$\text{loss} = -\log(\text{prob}) \rightarrow -(-ve) = +ve$$

loss  $\rightarrow$   $G_9$   
 $\rightarrow$   $G_{10}$   
 $\rightarrow$   $G_{11}$   
 $\rightarrow$   $G_{12}$

$$Loss \Rightarrow [G_0 + + - + + - + G_{15}]$$

Pytorch

optimizer.zero()

loss → →  
loss.backward()  
optimizer.step

$$W = \frac{W + \boxed{\nabla W}}{\text{optimizer.step}}$$

Loss  $\rightarrow$  - Optimization.

$$\begin{array}{c}
 - \\
 G_{0-15} \times \textcircled{\nabla} \\
 + \\
 G_{1-15} \times \nabla \\
 \vdots \\
 G_{2-15} \times \nabla \\
 \vdots \\
 \vdots \\
 G_{15-15} \times \nabla
 \end{array}$$

$\nabla = \nabla$  of policy

$\nabla$  of  $\text{prob}(a/s) \leftarrow$  Stochastic Aberration

$$\log(\text{prob}(a/s))$$

$$T = \frac{S_0 \quad S_1 \quad S_2 \quad \dots \quad S_{15}}{P_0 \times P_1 \times P_2 \times \dots \times P_{15}} \leftarrow \underline{0.02}$$

$$\log(P_0) + \log P_1 + \log P_2 +$$

$$\log(P_1 \times P_2 \times P_3) \rightarrow \underline{\log P_1} + \underline{\log P_2} + \underline{\log P_3}$$